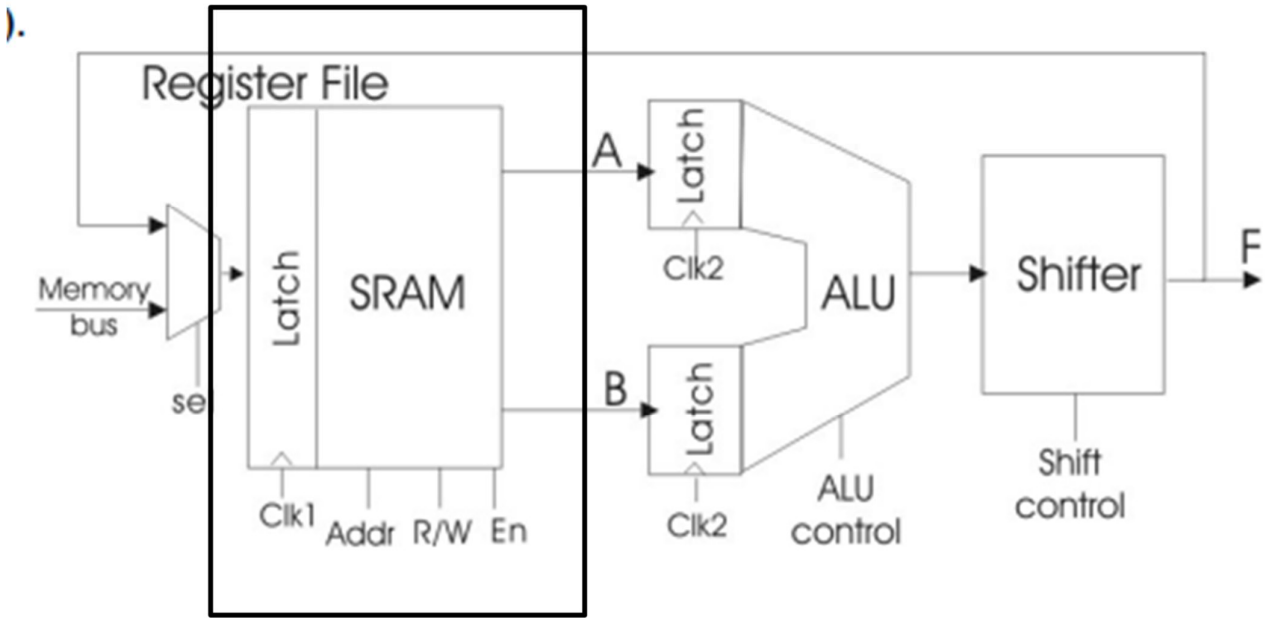
VLSI final project : S-Ram







John Najar , Wajdi Abo-Rees , Shaker Amon Mohamed Sawaed



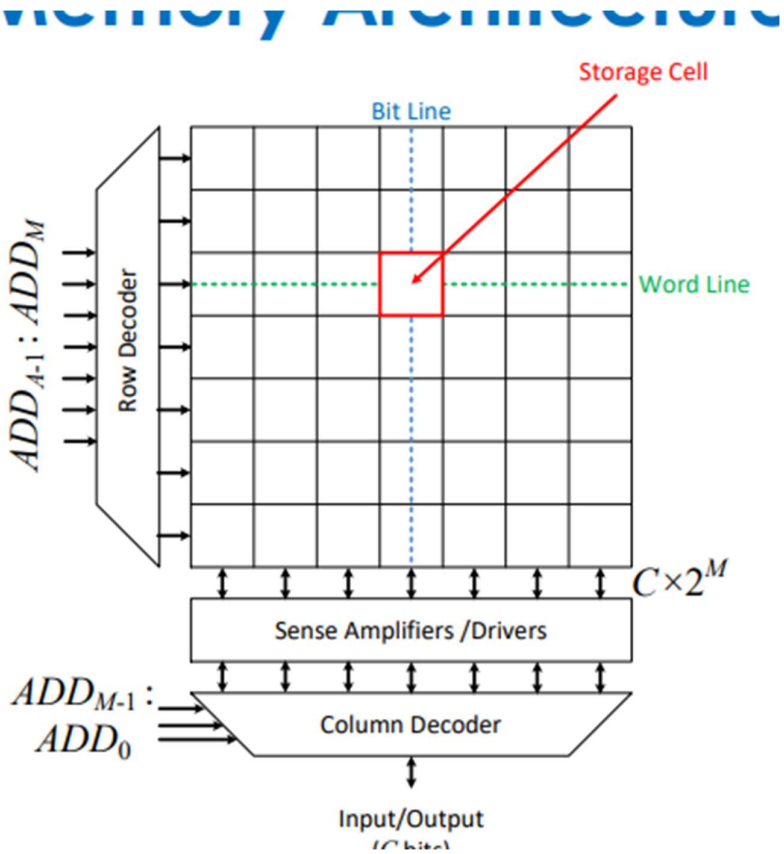
06/02/2025

Overview:

SRAM (Static Random-Access Memory) is a high-speed, volatile memory commonly used in VLSI for applications like CPU caches, register files, and buffers. It stores data using a 6-transistor (6T) design with cross-coupled inverters, eliminating the need for refreshing like DRAM. SRAM is faster and more reliable but consumes more static power and has a lower density due to its larger cell size. Despite its higher cost, it is essential in performance-critical systems for its speed and efficiency.

And our project we going to build a 8x8 S-ram with word of one byte (8 bits).

S-RAM architecture:



***Note : we used the write read as sense amplifier .**

Summary

VLSI Final Project - 8x8 SRAM Design

In this project, we designed and implemented an 8x8 Static RAM (SRAM) array using a 6T cell architecture. The goal was to develop a fully functional memory system capable of performing reliable read and write operations, while optimizing layout area, power consumption, and performance.

Key Achievements:

- 6T SRAM Cell Design: Built and tested a single memory cell for stable data storage.
- Memory Array (8x8): Constructed an 8-word, 8-bit memory (64 cells total).
- Row Decoder & Word Line: Designed a 3-input, 8-output row decoder to select word lines.
- **Bit Line Read/Write Operations**: Ensured stable voltage transitions and proper data retention.
- **Layout Optimization**:
  - Total area: 77 × 53.94 ≈ 4158 square units
  - Metal 2 used to avoid routing congestion.
  - Cell flipping for efficient power distribution (shared VCC and GND).
- **Performance:**
  - Rise/Fall time: ~34 ps
  - Propagation delay: ~95.8 ps
  - Stable write-to-read recovery time to prevent data corruption: ~100 ps
  - Max frequency of the system : Fmax = 1/ 100ps= 10 [Ghz] .
  - Number of Transistors : 594 .

Tasks :

1. **Building one memory array**
    a. **Building one cell**
        i. **Cell schematics**
        ii. **Cell layout**
    b. **Stack of 8 cells –WORD**
    c. **Building the entire array of 8x8 (8 stacks of 8 bits = 64 cells)**
2. **Building Row Mux (3 inputs and 8 outputs) :**
    a. **One output of 3 inputs and one output .**
        i. **Schematics**
        ii. **Layout**
    b. **Multiply it for 8 outputs.**

3. **Integrate the entire SRAM Schematics using both the mux and the memory array of 8X8.**
4. **Building the project final report .**

Building Row Mux (3 inputs and 8 outputs) :

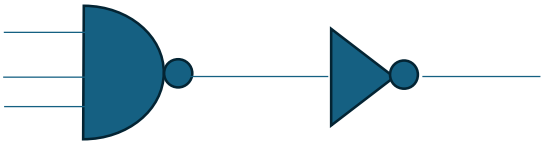
1. **One output of 3 inputs and one output .**

Comparison of Two 3-Input AND Gate Designs

The first design uses a single 3-input NAND gate followed by one inverter, implementing the logic:

$$AND_3 = NOT\ (\ NAND_3)$$

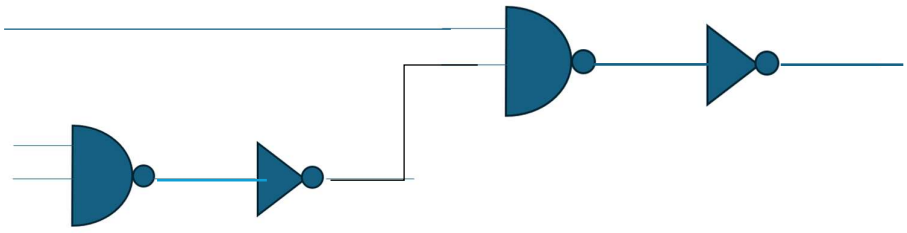
The inverter has a standard width, where NMOS and PMOS . This design is simple, has a lower transistor count, and consumes less power, making it suitable for applications where simplicity and efficiency are priorities. However, it suffers from slower operation due to the high resistance in the stacked NMOS transistors within the NAND gate.



The second design uses two 2-input NAND gates and two inverters to implement the same logic:

$$AND_3 = NOT\ (\ NAND_2(NOT\ (\ NAND_2)))$$

The first inverter has the same sizing as the first design, but the final inverter is larger, with and, to drive heavier loads. This design is faster and provides better drive strength due to reduced NMOS stacking and the additional inverter stage. However, it comes at the cost of higher power consumption, increased transistor count, and a more complex layout. While the first design is ideal for low-power, simple circuits, the second is better for speed-critical or high-load applications.

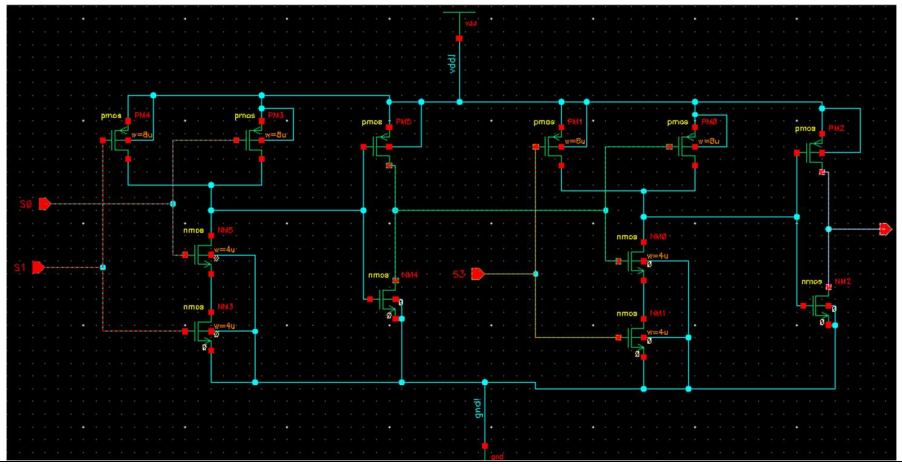


Conclusion:

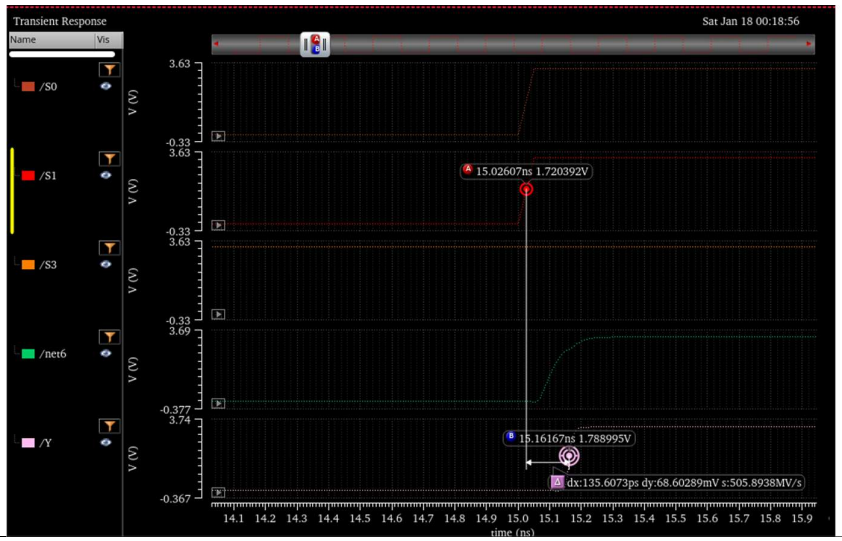
- **Design 1**: For simplicity and low power.

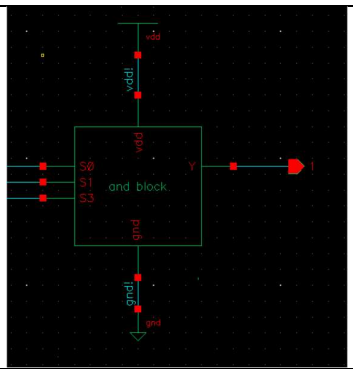
- **Design 2**: For speed and high load-driving needs. We going to pick this one .

And- 3 input one output

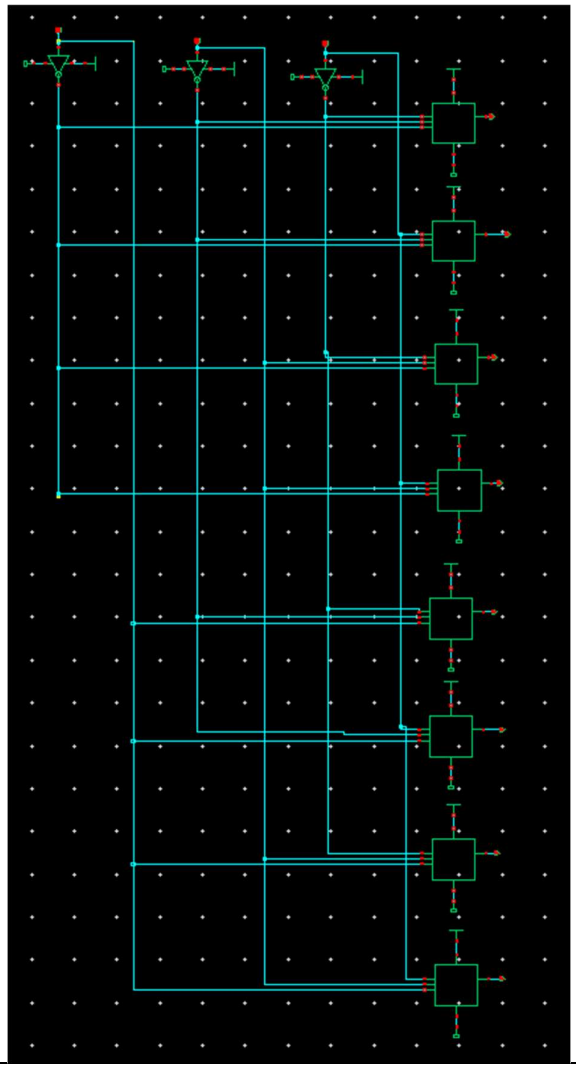
Schematics	

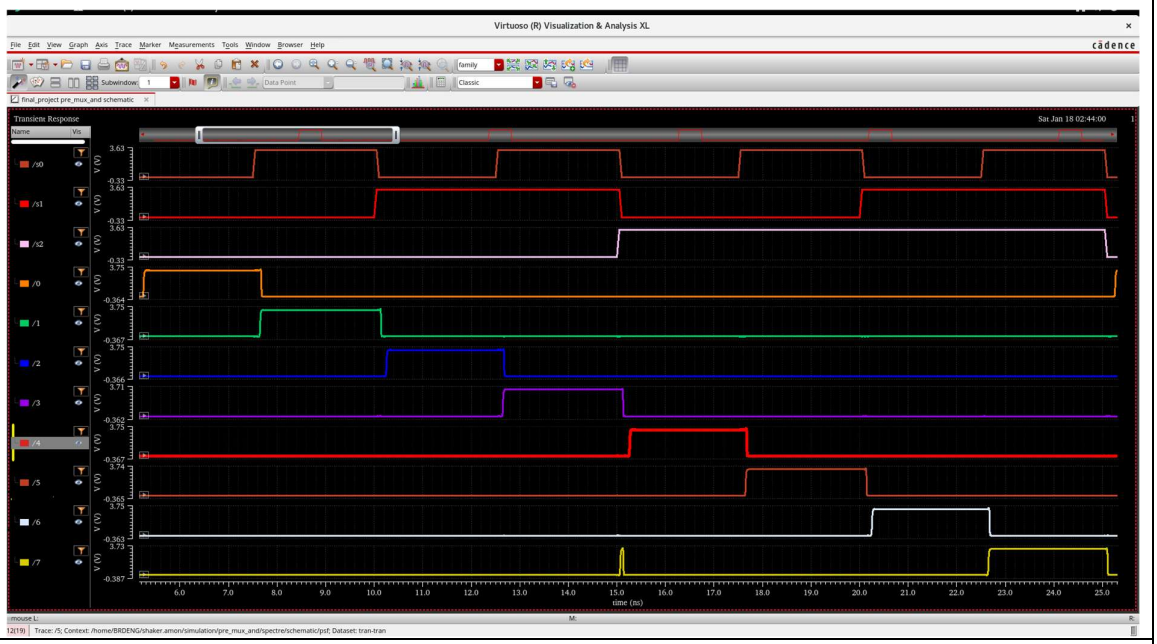
Stumulis	Open · stumulis_3nand.txt · Save
	`simulator lang=spectre`
	`vdd(vdd! 0 ) vsource dc=3.3`
	`gnd(gnd! 0 ) vsource dc= 0`
	`V1(S0 0) vsource type=pulse val0=0 val1=3.3 delay=0 rise=0.05n fall=0.05n width= 2.5n period= 5n`
	`V2(S1 0)  vsource type=pulse val0=0 val1=3.3 delay=5n rise=0.05n fall=0.05n width= 5n period= 10n`
	`V3(S3 0) vsource type=pulse val0=0 val1=3.3 delay=5n rise=0.05n fall=0.05n width= 10n period= 20n`

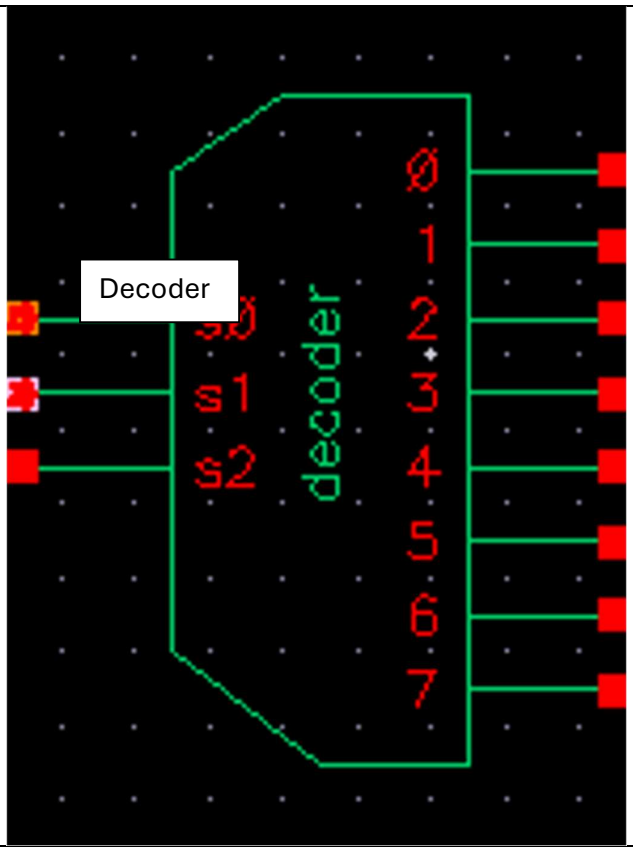
Wave	
Symbol	

Decoder

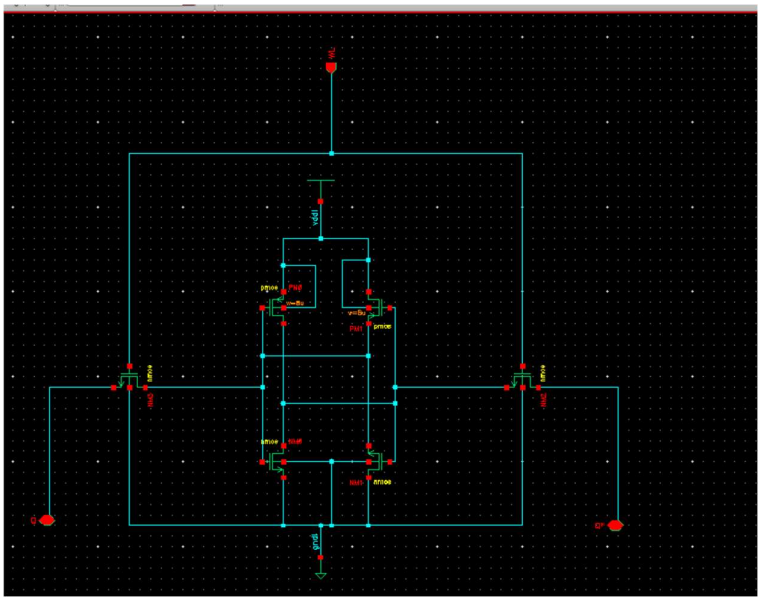
Schematics	
Stumulis	stimulus_nand.txt × stumulis_3nand.txt × stumulis_mux.txt simulator lang=spectre vdd(vdd! 0) vsource dc=3.3 gnd(gnd! 0) vsource dc= 0 V1(s0 0) vsource type=pulse val0=0 val1=3.3 delay=2.5n rise=0.05n fall=0.05n width= 2.5n period= 5n V2(s1 0) vsource type=pulse val0=0 val1=3.3 delay=10n rise=0.05n fall=0.05n width= 5n period= 10n V3(s2 0) vsource type=pulse val0=0 val1=3.3 delay=15n rise=0.05n fall=0.05n width= 10n period= 20n

Wave	 Virtuoso (R) Visualization & Analysis XL File Edit View Graph Axis Trace Marker Measurements Tools Window Browser Help cadence Transient Response Sat Jan 18 02:44:00 /s0 /s1 /s2 /0 /1 /2 /3 /4 /5 /6 /7 time (ns)
Symbol	

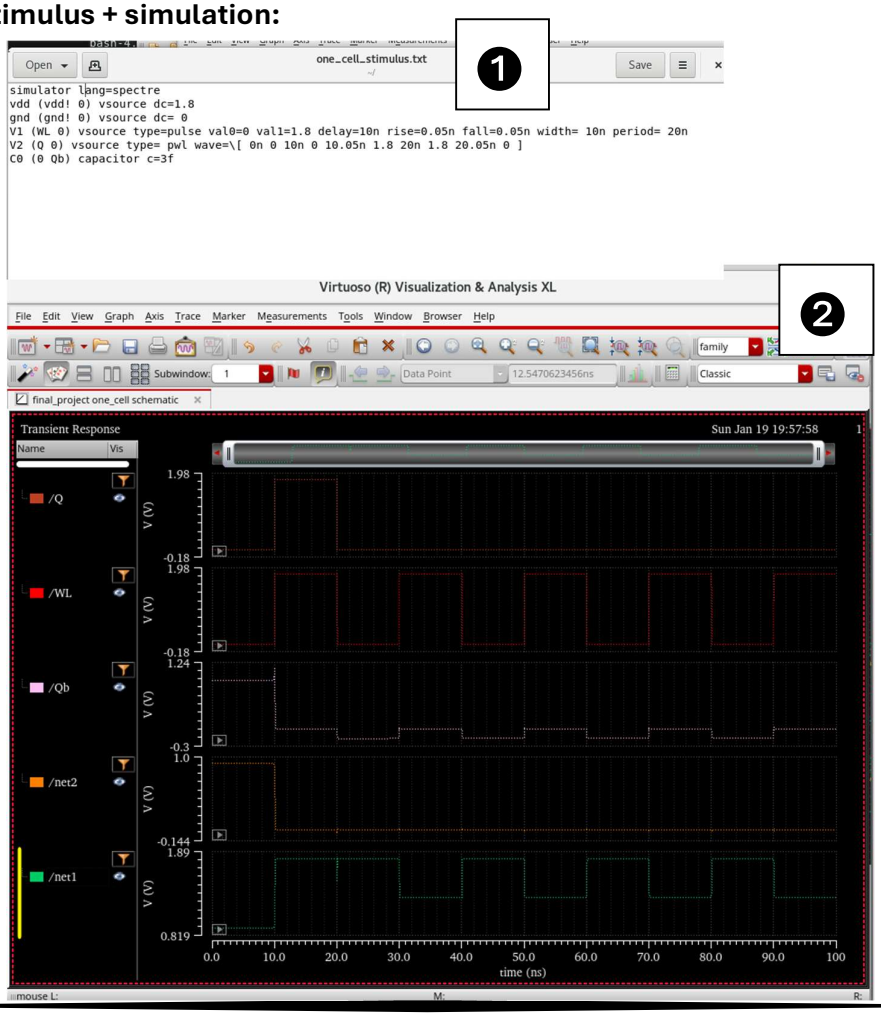

Decoder
s0
s1
s2
decoder
0
1
2
3
4
5
6
7

Building one memory array

1. Schematic 6T CELL:

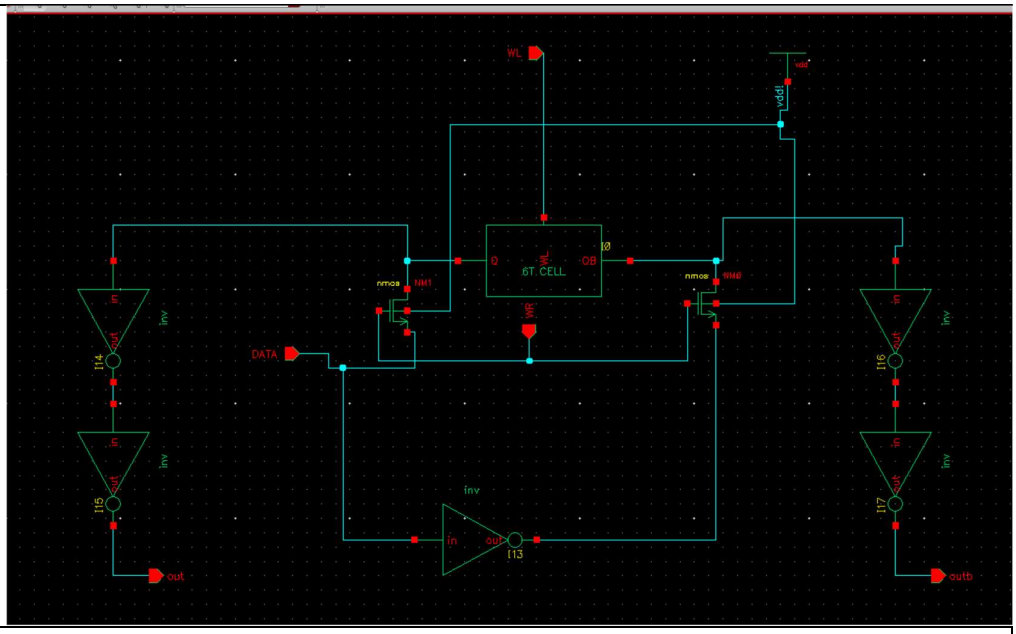
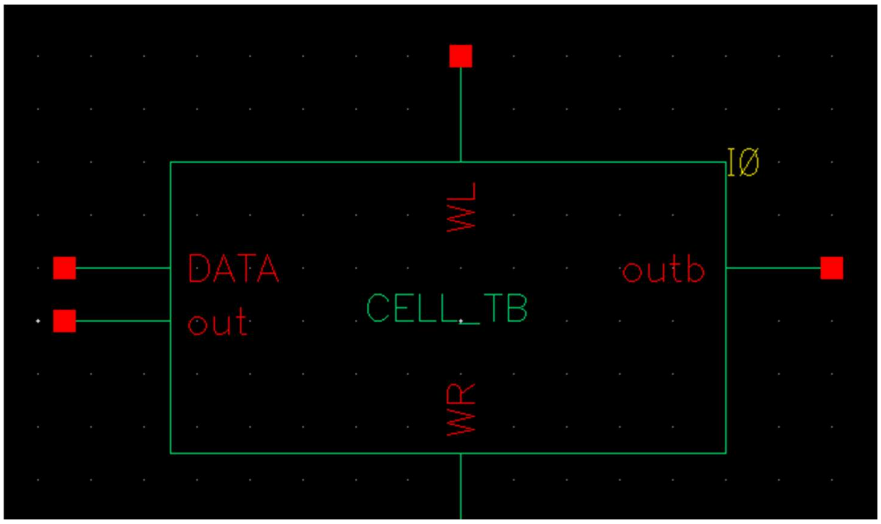


2. stimulus + simulation:



6T Cell test bench

In the standard 6T SRAM cell, the input (write) and output (read) share the same bit lines. This creates conflicts during read and write operations, as the cell cannot simultaneously drive and be driven by the same line. Additionally, the shared lines can disturb the stored data during reads or make it difficult to fully overwrite data during writes. To address these issues and enable reliable independent read and write operations, a new cell design was developed, incorporating additional circuitry to separate the input and output paths. This modification ensures stable functionality and improves overall performance.

Schematic	
Symbol	

Stimulus	simulator lang=spectre vdd(vdd! 0) vsource dc=1.8 gnd(gnd! 0) vsource dc=0 V1(DATA 0) vsource type=pulse val0=0 val1=1.8 delay=0 rise=0.05n fall=0.05n width= 25n period= 50n V2 (WR 0) vsource type=pulse val0=0 val1=1.8 delay=0n rise=0.05n fall=0.05n width= 50n period= 100n V3 (WRB 0) vsource type=pulse val0=0 val1=1.8 delay=50n rise=0.05n fall=0.05n width= 50n period= 100n
Wave	

- We have achieved out goal we can write and read from the cell without affecting the data inside the cell.

Testing section

In this section, we will test the integration of our design components to ensure proper functionality. This includes verifying the decoder's activation of word lines, the behaviour of BL and BLB during read and write operations, and the connection of the latch to the decoder for accurate data handling. These tests will confirm that all parts work together seamlessly.

❶ decoder with word line

i. **Build word line of 8 cell (stack)**
ii. **Integrate with decoder (building simulation)**

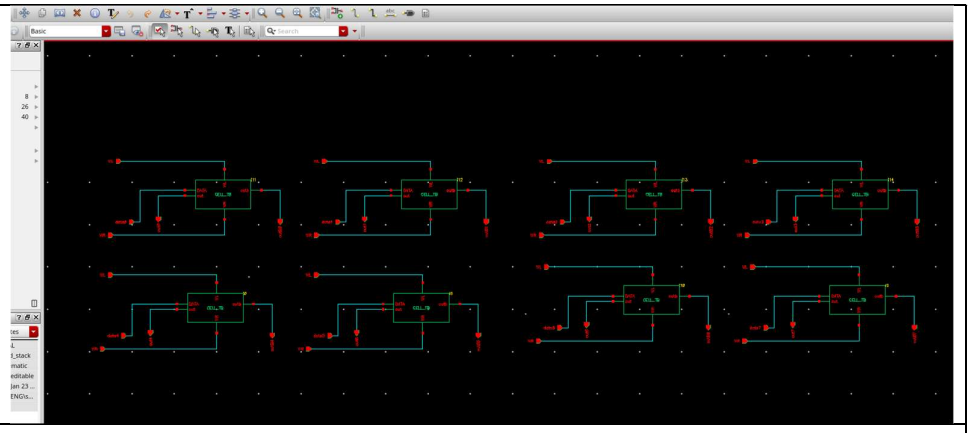
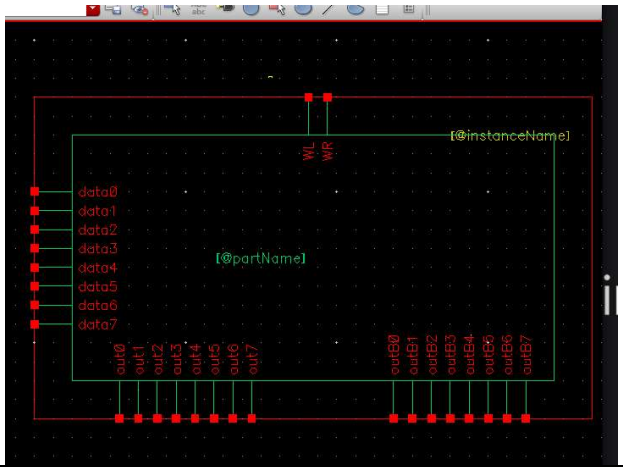
❷ BL & BLB

i. **Building stack 8 bit of one column**
ii. **Creating simulation**

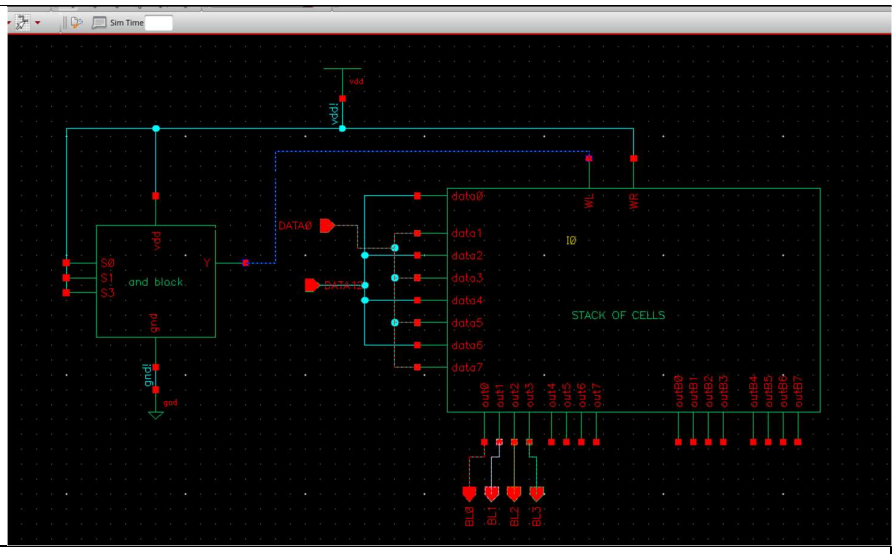
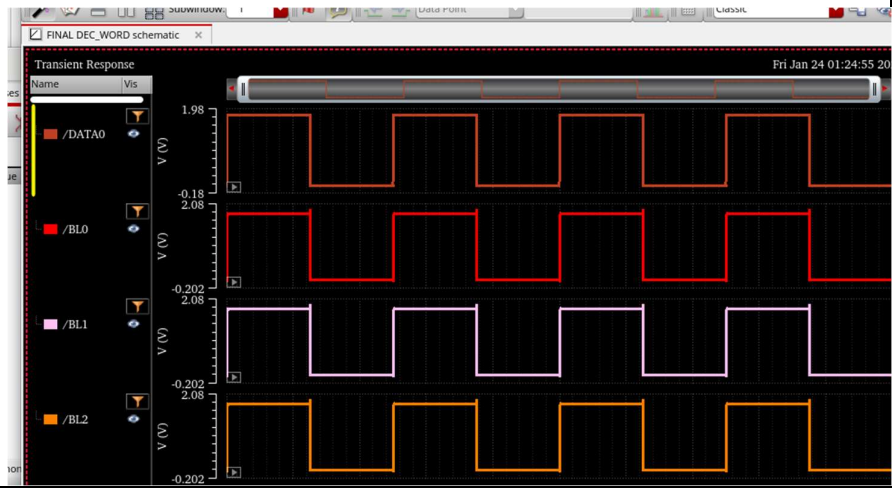
Testing the word line with Decoder

1. Building a word line stack

In this part we build a word line of 8 cells.

Schematics	
Symbol	

2. Decoder with word line(8 cells)

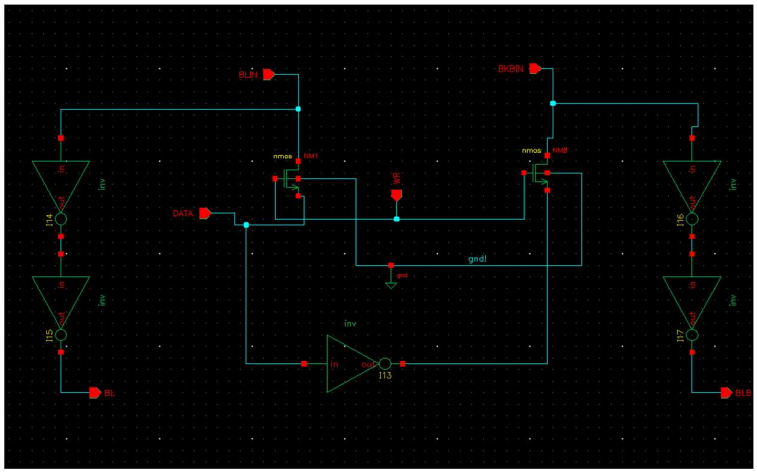
Schematics	
Stimulus	write_read_WORD.txt simulator lang=spectre vdd(vdd! 0) vsource dc=1.8 gnd (gnd! 0) vsource dc=0 V1 (DATA0 0) vsource type=pulse val0=0 val1=1.8 delay=0 rise=0.05n fall=0.05n width= 25n period= V2 (WR 0) vsource type=pulse val0=0 val1=1.8 delay=0n rise=0.05n fall=0.05n width= 10n period= 20 V3 (DATA12 0) vsource type=pulse val0=0 val1=1.8 delay=5n rise=0.05n fall=0.05n width= 25n period
wave	

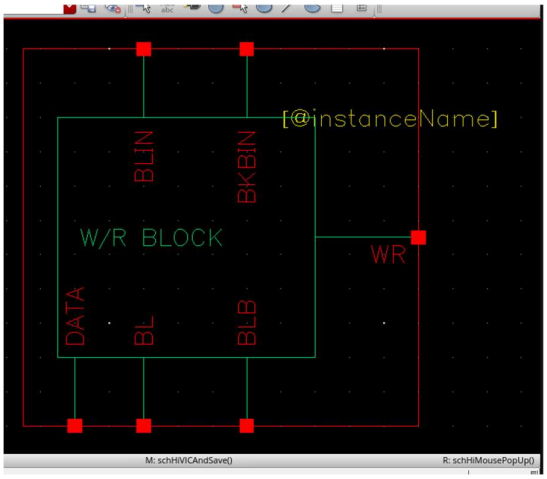
***in this simulation we proved that the word line work great with one stack (8 bits) .**

Testing the BL & BLB

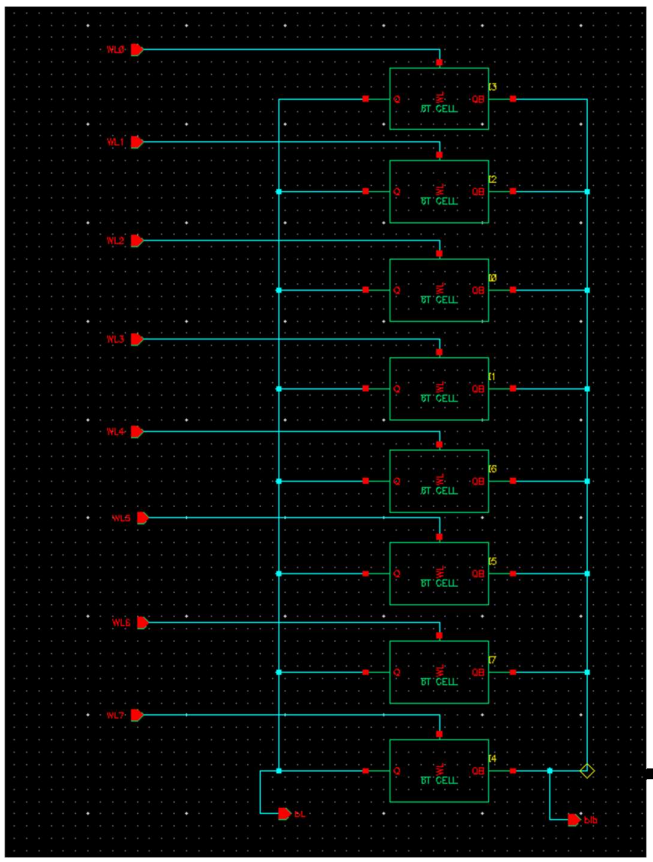
This section focuses on the testing and evaluation of the bit line within the 6T SRAM cell. It involves analyzing its performance during read and write operations, ensuring signal integrity, and verifying proper functionality under varying conditions. Specific attention is given to the voltage levels, timing requirements, and the overall impact on data stability.

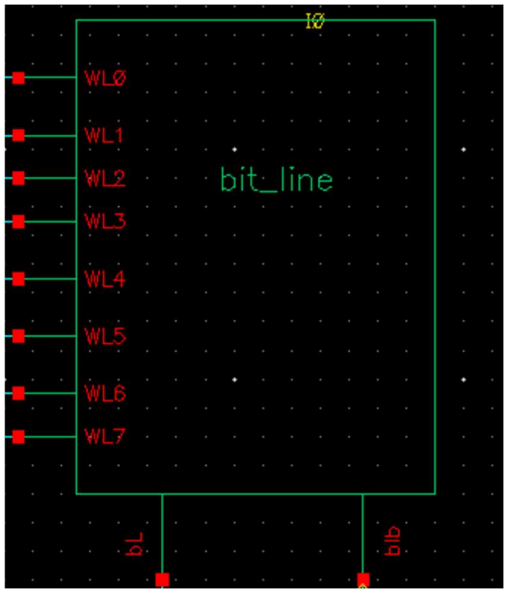
- First we build a block for reading and writing from the cells in the bite line (we used something similar for the one cell 6T Test bench)





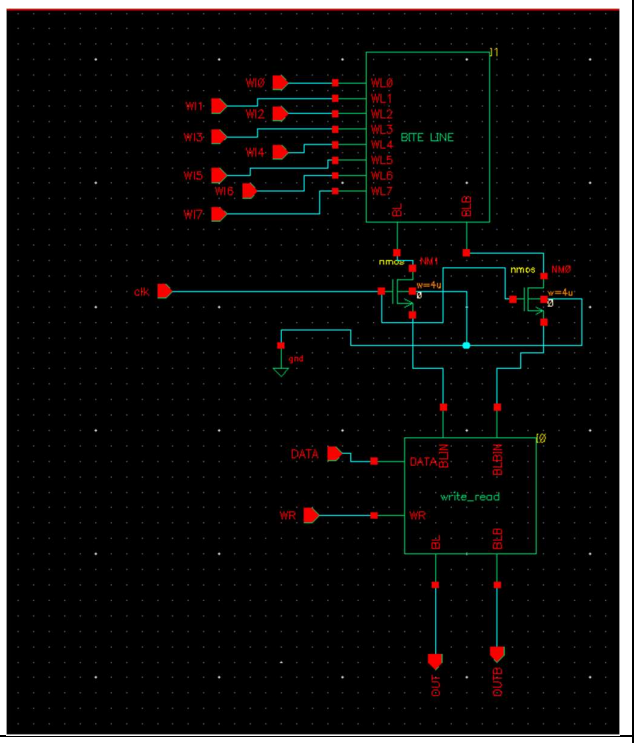
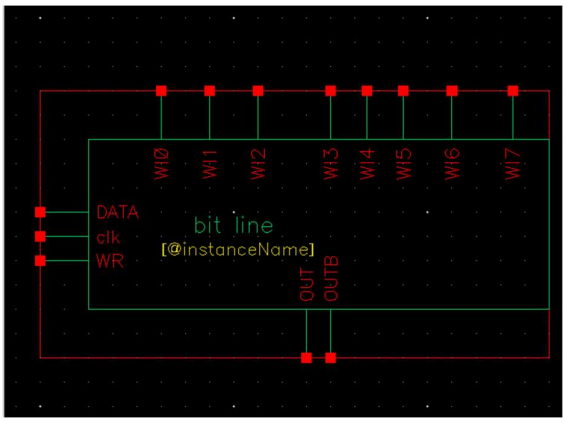
- **Building a bite line of 8 cells**



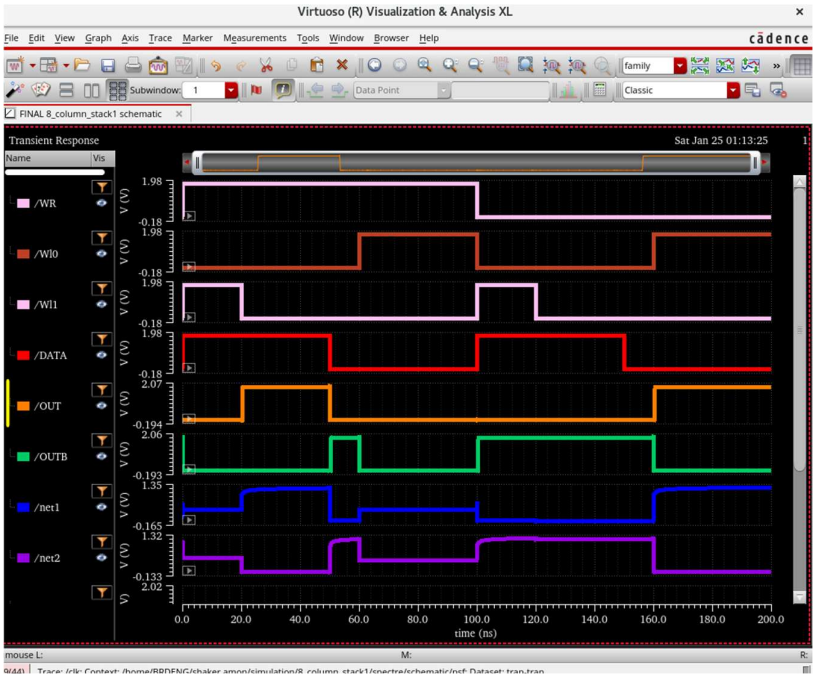


"Bit Line Test Bench and Simulation"

After constructing the test bench for the bit line, the next step involved creating the bit line simulation. This simulation was designed to verify the behaviour of the bit line during both read and write operations. It focused on assessing voltage transitions, stability under various load conditions, and ensuring the expected functionality aligned with design specifications. Key metrics such as delay, noise margin, and signal integrity were analysed to validate the bit line's performance.

Schematics	
symbol	

Stimulus	Open ▾ bite_line.txt ~/
	// SRAM Stimulus File (Spectre)
	simulator lang=spectre
	// Power Supply Sources
	vdd (vdd! 0) vsource dc=1.8
	gnd (gnd! 0) vsource dc=0
	
	// Control Word Lines (WL1-WL7)
	VWL0 (Wl0 0) vsource type=pulse val0=0 val1=1.8 delay=60n rise=0.05n fall=0.05n width=40n period=100n
	VWL1 (Wl1 0) vsource type=pulse val0=0 val1=1.8 delay=0n rise=0.05n fall=0.05n width=20n period=100n
	//VWL2 (Wl2 0) vsource type=pulse val0=0 val1=1.8 delay=20n rise=0.05n fall=0.05n width=5n period=100n
	//VWL3 (Wl3 0) vsource type=pulse val0=0 val1=1.8 delay=30n rise=0.05n fall=0.05n width=5n period=100n
	//VWL4 (Wl4 0) vsource type=pulse val0=0 val1=1.8 delay=40n rise=0.05n fall=0.05n width=5n period=100n
	//VWL5 (Wl5 0) vsource type=pulse val0=0 val1=1.8 delay=50n rise=0.05n fall=0.05n width=5n period=100n
	//VWL6 (Wl6 0) vsource type=pulse val0=0 val1=1.8 delay=70n rise=0.05n fall=0.05n width=5n period=100n
	//VWL7 (Wl7 0) vsource type=pulse val0=0 val1=1.8 delay=80n rise=0.05n fall=0.05n width=5n period=100n
	
	clk(clk 0) vsource dc = 1.8
	
	// Write Enable (WR)
	VWR (WR 0) vsource type=pulse val0=0 val1=1.8 delay=0n rise=0.05n fall=0.05n width=100n period=200n
	
	// Data Input (DATA)
	VDATA (DATA 0) vsource type=pulse val0=0 val1=1.8 delay=0n rise=0.05n fall=0.05n width=50n period=100n
	
	// End of File

wave	

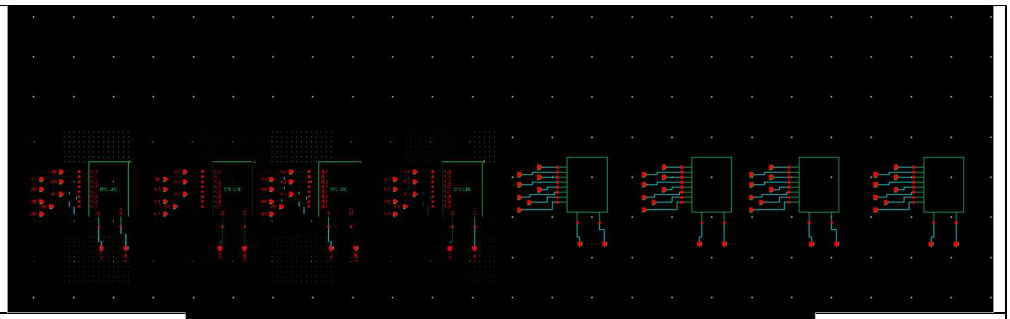
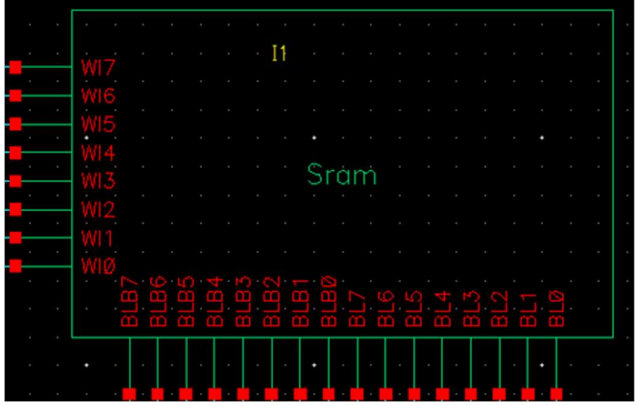
Conclusion:

In this section, we examined how the components interact and assessed potential issues, including fan-out problems, delay times, or other performance concerns. Based on our analysis and simulation results, we successfully demonstrated that the system operates as intended, with no critical issues identified. This confirms the proper functionality of the design

Integration

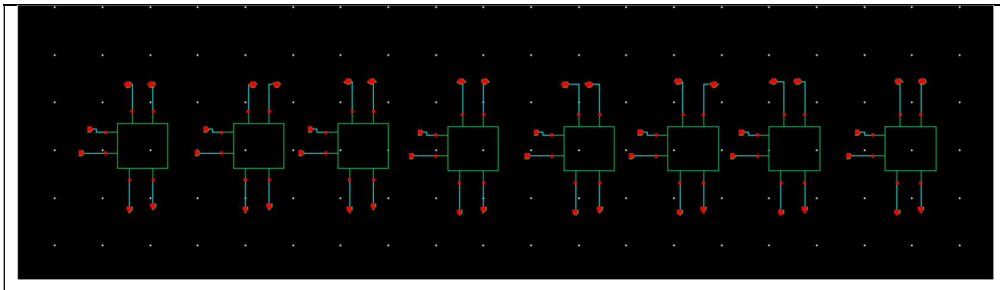
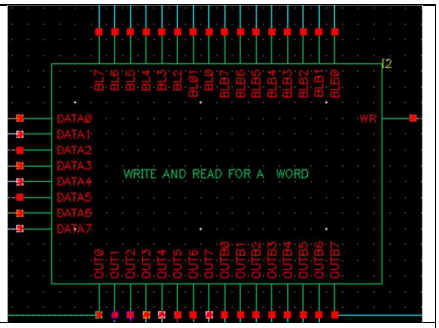
In this section, we integrate the components to construct the complete SRAM. This process involves assembling the individual modules, such as the bit lines, word lines, and control logic, ensuring seamless interaction between them to achieve the desired functionality of the SRAM system.

- Build the 64 cells, 8x8 bits:
  We build the 64 cell of bits 8x8 array ram .

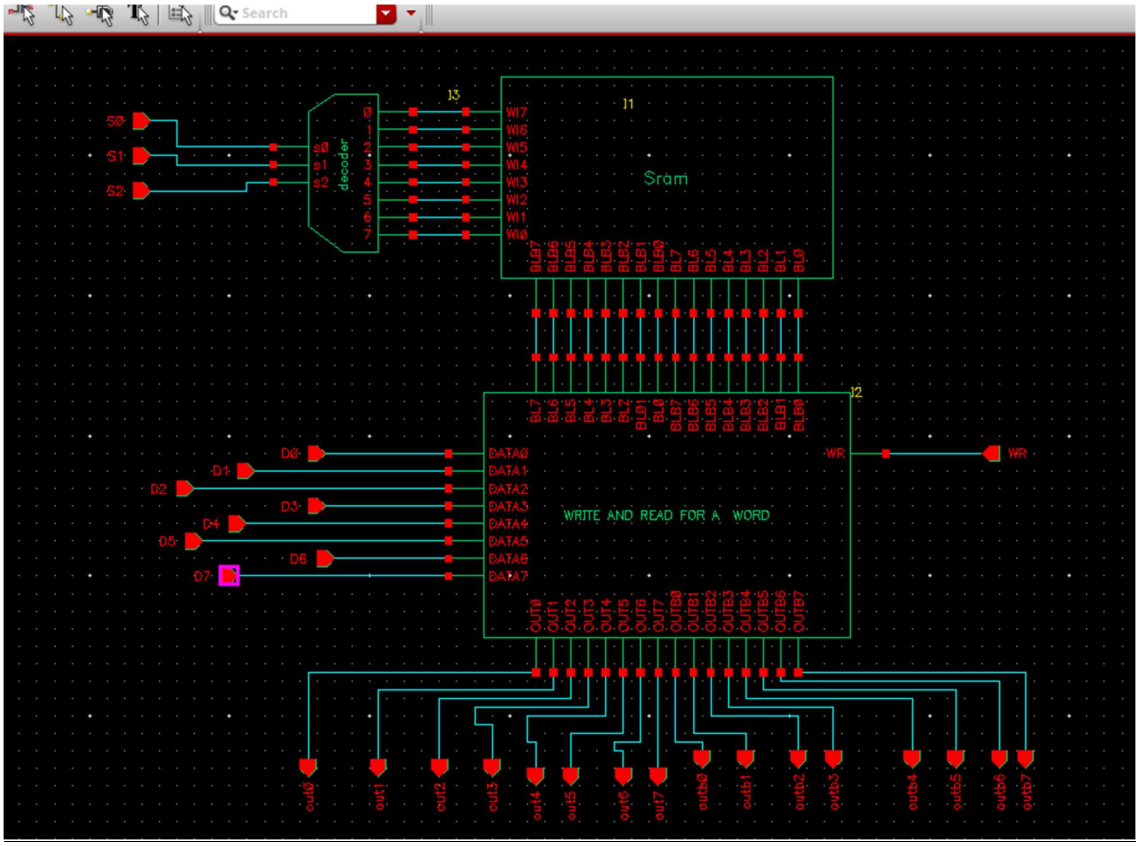
Schematics	
Symbol	

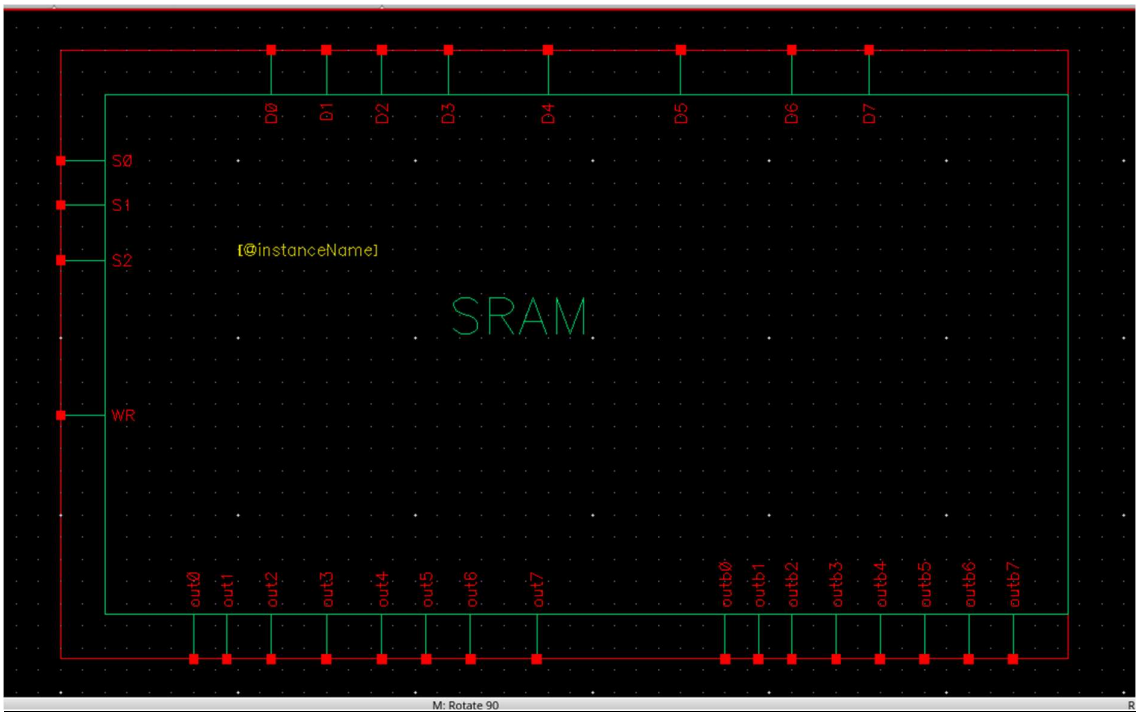
- **Write read block:**

  We need to add read write block for the entire lines , we build 8 in _ 8out block of write and read block .

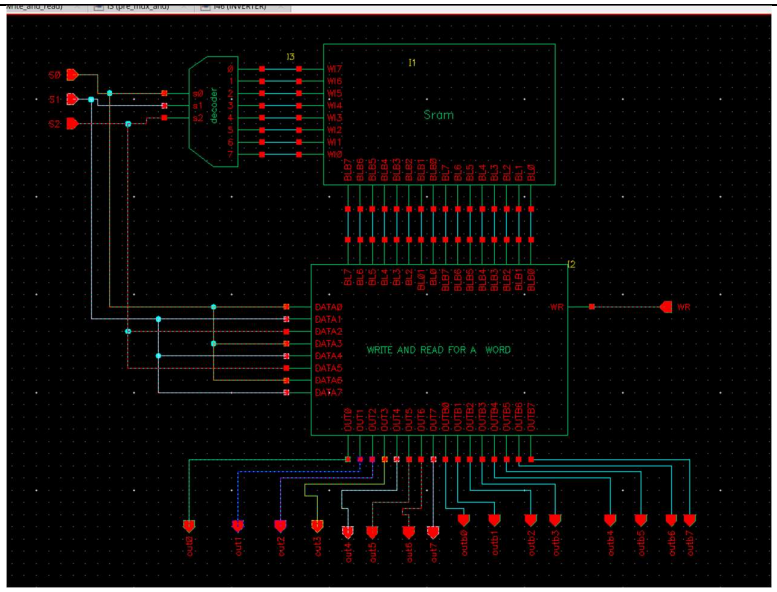
	
schematic	**symbol**

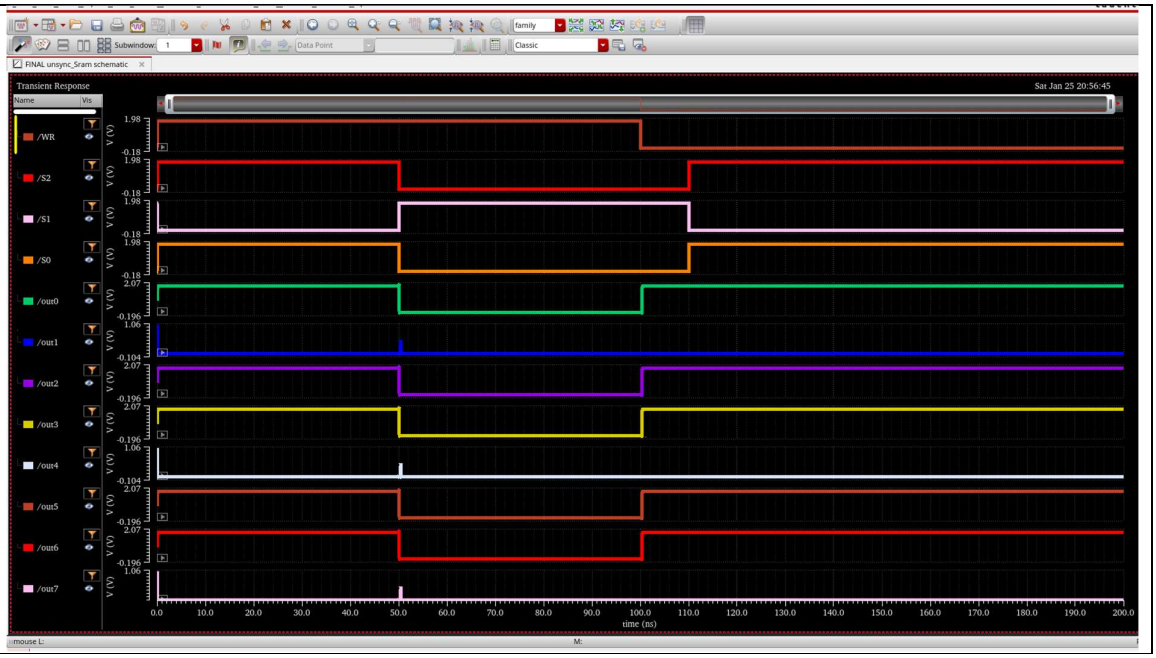
SRAM- final Symbol and Schematics



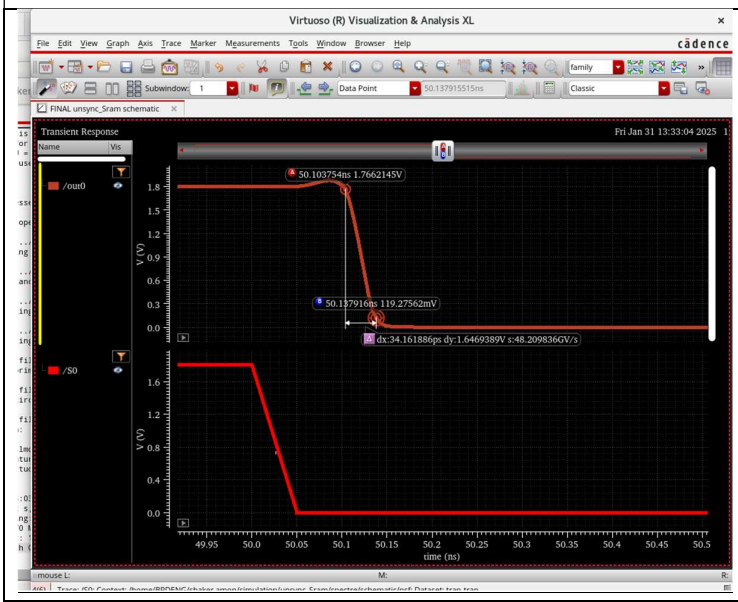
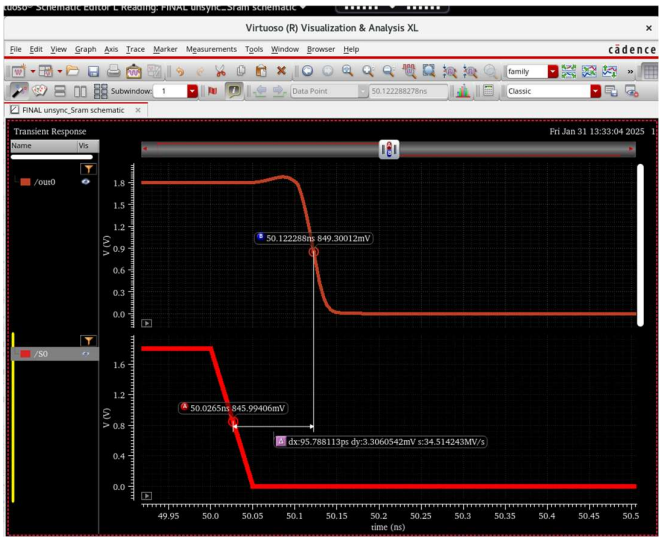


SRAM With test bench

schematic	
stimulus	Open unsync_ram.txt Save simulator lang=spectre vdd (vdd! 0) vsource dc=1.8 gnd (gnd! 0) vsource dc=0 VWR (WR 0) vsource type=pulse val0=0 val1=1.8 delay=0n rise=0.05n fall=0.05n width= 100n period= 200n V0(S0 0) vsource type=pwl wave=\[0n 0 0.05n 1.8 50n 1.8 50.05n 0 110n 0 110.05n 1.8] V1(S1 0) vsource type=pwl wave=\[0n 1.8 0.05n 0 50n 0 50.05n 1.8 110n 1.8 110.05n 0] V2(S2 0) vsource type=pwl wave=\[0n 0 0.05n 1.8 50n 1.8 50.05n 0 110n 0 110.05n 1.8]

wave	

Other tests

Raise / fall time	Propagation delay
	

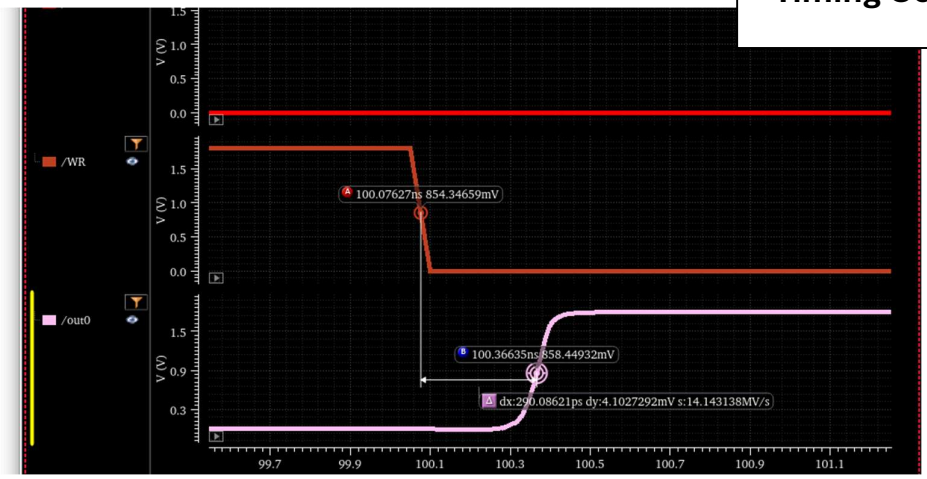
RT = FT = 34 [Ps]

propagation delay = 95.8 [Ps]

Timing Constraints (Write-to-Read Recovery)

Another critical consideration in SRAM design is ensuring adequate write-to-read recovery time. This interval allows the newly written data to fully settle in the 6T cell and ensures that the bit lines are restored to the correct voltage levels before initiating a read. If the read operation commences too soon, the cell might not have stabilized, risking incorrect or incomplete data capture. Therefore, careful validation of the minimum recovery time between the de-assertion of the write enable signal and the initiation of the read cycle is essential to guarantee reliable SRAM performance under various operating conditions.

Timing Constraints = 100 [Ps]

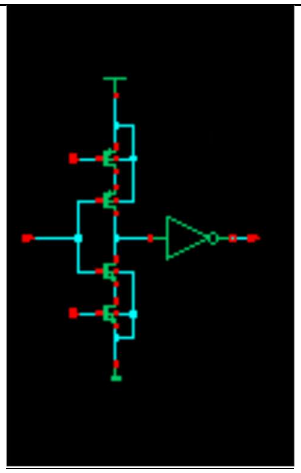
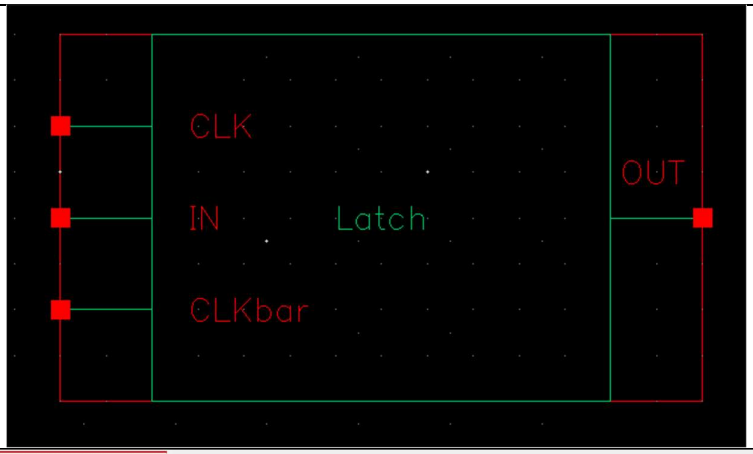
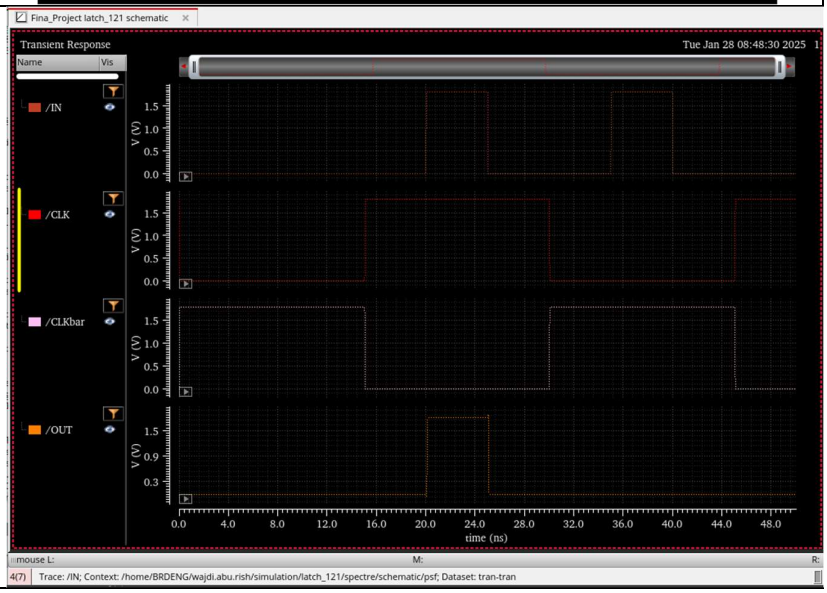


Max Frequency

Fmax = 1/ 100 ps = 10 [Ghz]

Your SRAM operates at high speed due to two key optimizations: a **fast 3-to-8 decoder** and the use of inverters as sense amplifiers in the read/write circuit. The decoder is designed with efficient NAND/AND logic, reducing propagation delay and ensuring rapid word-line activation. Instead of traditional sense amplifiers, **inverters amplify bit-line signals**, accelerating voltage transitions and improving read performance. This design **choice minimizes capacitance** loading and reduces overall delay, allowing the system to achieve faster memory access and a higher operating frequency.

Latch

Schematics	
Symbol	
Waves	

***although inside the ram we do not add the latch but in the entire CPU system it would be part connecting the inputs of the SRAM with the other systems such as ALU .**

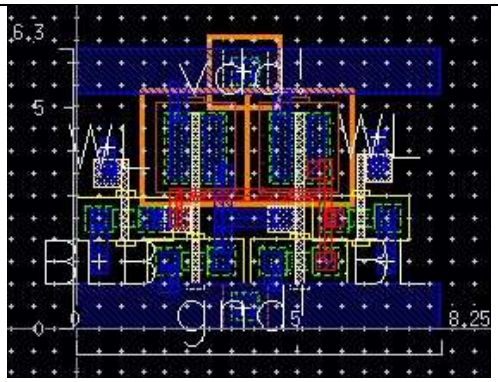
Layout

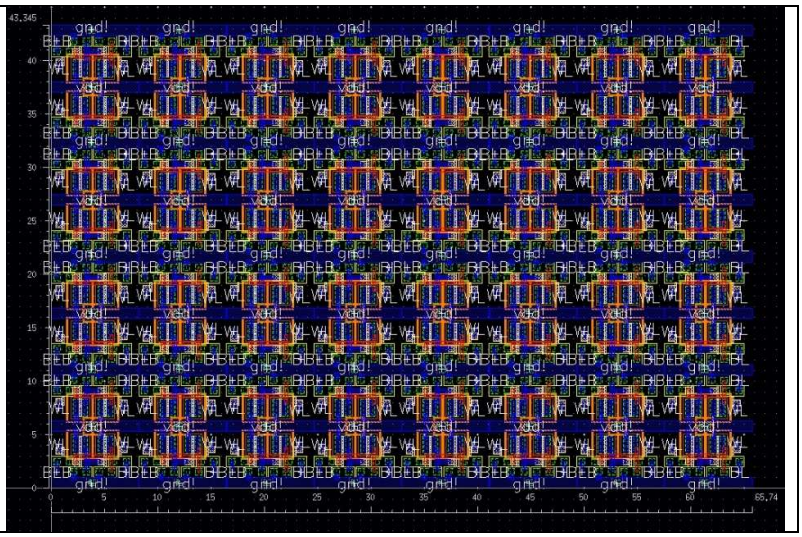
In this section, we present the layout designs for the components required in the SRAM implementation. The layouts include the following:

- **NAND Gates and Inverters**: These components are used in the decoder circuit to facilitate address selection.
- **6T SRAM Cell Layout**: This layout represents the core memory cell, including six transistors (4 NMOS and 2 PMOS) configured to store a single bit of data.
- **Latch Layout**: The latch is designed to stabilize and retain data during read and write operations.

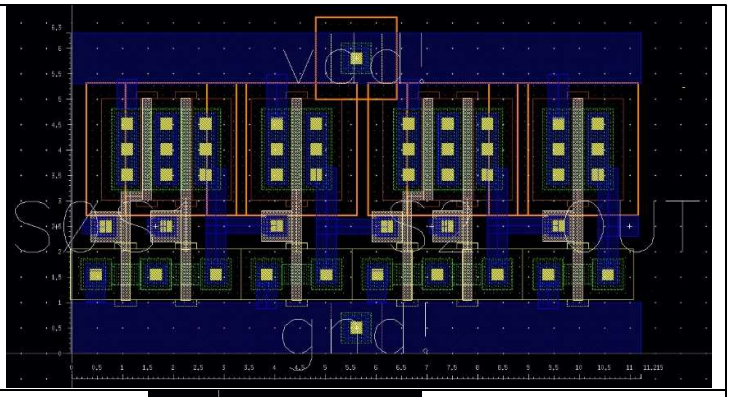
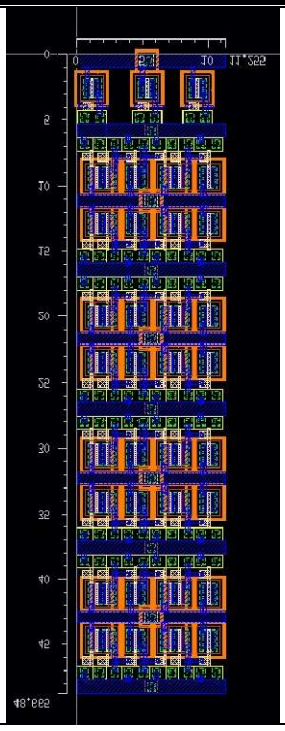
  The layouts are carefully optimized to achieve high performance, minimal area, and reduced power consumption, adhering to standard design rules.

1. Cell Array layout:

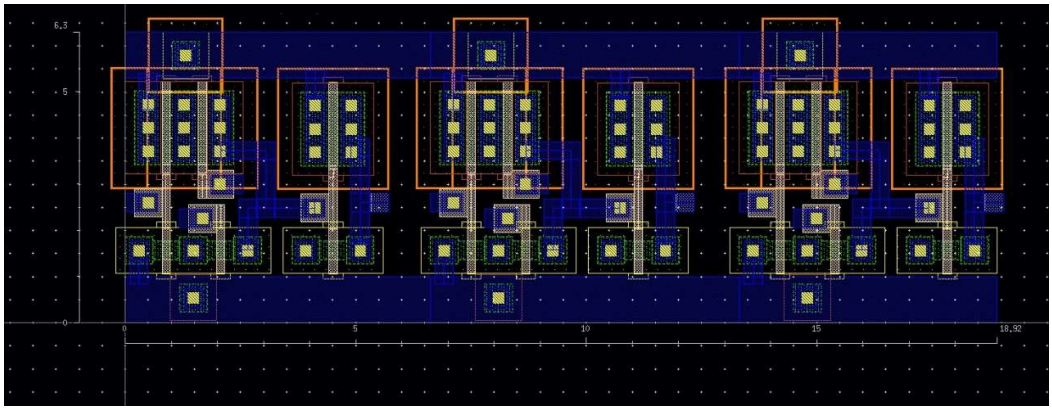
One cell	
64 cells	



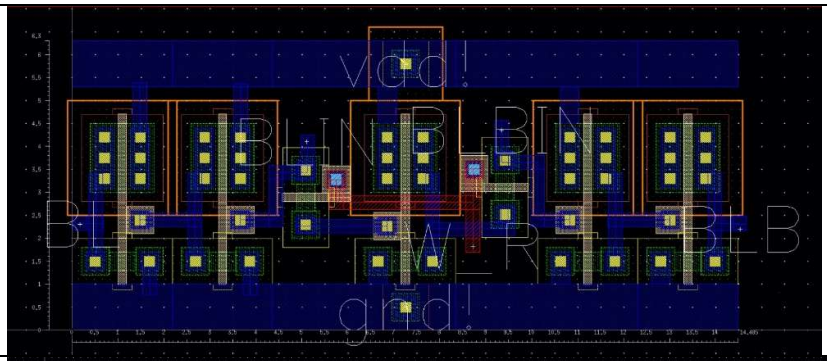
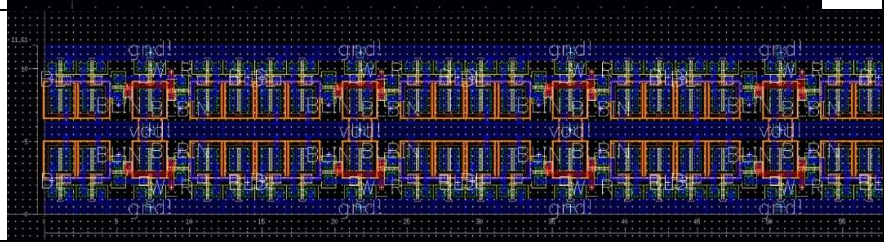
2. Decoder layout:

And 3 in one out	
Decoder layout	

3. **Latch:**

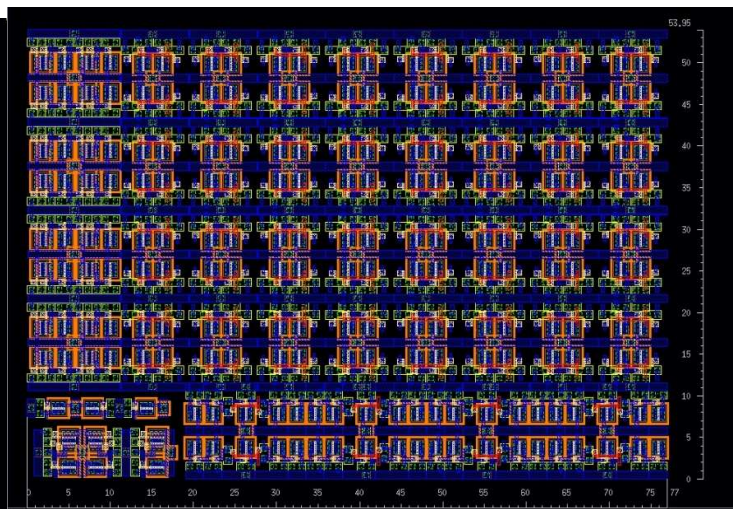


W/R function :

One output	
8 w/r function	

Full system





Conclusions - For schematics

This project successfully demonstrated the design and implementation of an 8x8 SRAM using a 6T cell architecture. Through systematic development, we tackled various challenges, including designing individual memory cells, integrating them into an array, and verifying their functionality with robust test benches and simulations. The key achievements of this project include:

6T Cell and Array Construction

- Designed and tested a single 6T SRAM cell.
- Scaled up to an 8×8 array (64 cells) organized into 8-bit words.

Decoder and Word-Line Integration

- Built a 3-input NAND-based row decoder for 8 word-lines.
- Verified correct selection of word-lines in simulation (only one word-line asserted at a time).

Bit-Line Testing

- Developed a bit-line block to handle read/write signals for 8-bit data.
- Simulations confirmed correct voltage transitions, stable read, and reliable write operations.

Timing and Performance

- Rise/Fall Times (RT/FT) measured at ~34 ps each.
- Propagation Delay (decoder-to-output) measured at ~95.8 ps.
- Ensured adequate write-to-read recovery time to avoid data corruption ~ 100ps.

Overall Integration

- Combined decoder, bit-lines, and 64 cells into a complete SRAM.
- Final tests showed consistent data storage and retrieval under normal conditions.(
- Number of the transistors **: 594** transistors = latch(18) + decoder(96)+w/r(96) + 64 cell array(384).

Layout

Total Area: The full system layout has an area of 77 × 53.94 ≈ 4158 [um^2].

Metal Layer Usage: To save a length of 6.4 units, Metal 2 was used.

This decision was made because Metal 1 alone could not accommodate the required routing without congestion or design rule violations.

Cell Flipping for Space Optimization:

The cells were flipped in the layout to align four rows with a common GND and VCC.

This helps in reducing interconnect complexity and improving power distribution.

Outcome

The project met its objectives by successfully implementing a fully functional SRAM with 8x8 memory cells. The simulations proved the system's ability to handle read and write operations reliably and efficiently. This project provided valuable hands-on experience in VLSI design, from conceptualization to verification, reinforcing our understanding of SRAM architecture and its applications.
Future Improvements

Conclusions table

Rise/Fall Times	34 ps
Propagation Delay	95.8 ps
write-to-read recovery time	100ps
Full system Number of the transistors	**594**
Area of the full system	77 × 53.94 ≈ 4158 square [um^2]